Feed-forward networks in PyTorch

Lecture 5 — Real models, real data, real choices

Dr Thomas Robinson

August 10, 2026

LSE SUMMER SCHOOL · 2026

Where are we?

AFTER THURSDAY'S LECTURE

Four days writing autodifferentiation by hand. **From today, you never have to again**

- (except maybe in an exam 😉)

Question for today:

How do we build neural networks at scale – in PyTorch, on real data, with a working classifier?

Concepts in practise

Moving from "understanding the algorithm" to "building a useful model".

After today, every remaining lecture uses PyTorch.

Warm-up · three questions from last time

≈ 5 MIN · SHOUT IT OUT

FROM LAST WEEK

1. One gradient-descent update. $w = 1$, $\frac{\partial L}{\partial w} = 0.5$, learning rate $\lambda = 0.1$. What is the new w?
2. What does a loss function measure — and which loss did we use for regression?
3. In which direction does backpropagation walk the computational graph, and what does it carry with it?

The weekend was long. Everything here is from Thursday's lecture!

Warm-up · answers

1. Step against the gradient, scaled by λ:

$$w_{\text{new}} = w - \lambda \frac{\partial L}{\partial w} = 1 - 0.1 \times 0.5 = 1 - 0.05 = 0.95$$

2. How wrong the model's predictions are on the data, as a single number — lower is better. For regression we used **mean squared error**.
3. **Backwards** — from the loss to the inputs, multiplying local derivatives along the way. That is the chain rule.

All three are about to become one line of PyTorch each.

Part 1 · From `Value` to PyTorch

What PyTorch gives us

PyTorch is the autodiff library you built — only:

- **Faster** — runs on GPU.
- **More general** — operates on arbitrary tensors, not just scalars.
- **Better engineered** — handles edge cases, numerical stability, parallelism.

The mental model is identical to your `Value` library. Same forward pass. Same backward pass. Same parameters, gradients, and updates.

Note

The point of last week was to demystify what PyTorch does. Now we let it do that work for us — and we focus on **design** instead of bookkeeping.

Tensors — the building block

You have met tensors already in the labs:

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])            # 1-D tensor (a "vector")
W = torch.randn(4, 3)                         # 2-D tensor (a "matrix"), random-init
y = x @ W.T                                   # matrix multiplication
print(y.shape)                                # torch.Size([4])
```

Tensor properties

- `.shape` — dimensions.
- `.dtype` — float32, int64, etc.
- `.device` — `cpu` or `cuda:0`.
- `.requires_grad` — should we track gradients?

OPERATIONS

- `@` or `torch.matmul` — matrix multiply.
- `+, -, *, /` — element-wise.
- `torch.exp`, `torch.log` — element-wise math.
- `.sum()`, `.mean()` — reductions.

Autograd — backpropagation, automatically

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 4 * x                    # y is a Value-like node — PyTorch builds the graph
y.backward()                          # backpropagate from y
print(x.grad)                         # → 16.0   (which is 3x² + 4 evaluated at x=2)
```

That is the whole API for autodiff in PyTorch.

- You can build any expression with tensors, it will automatically track the resulting graph.
- When you call `.backward()` on the scalar output tensor, you can then read off `.grad` from the inputs to see the results!
- This is exactly the algorithm you have been working on by hand.

Models, the PyTorch way — `nn.Module`

```python
import torch.nn as nn

class TurnoutModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3, 16)     # 3 inputs → 16 hidden neurons
        self.fc2 = nn.Linear(16, 1)     # 16 hidden → 1 output

    def forward(self, x):
        h = torch.relu(self.fc1(x))     # hidden layer with ReLU
        return self.fc2(h)              # output (logit)
```

- `nn.Linear(k, h)` — a fully-connected layer with $k \cdot h$ weights and h biases. Initialised for you.
- `nn.Module` — is a base class that tracks parameters and provides key methods like `.parameters()`, `.train()`, `.eval()`, `.to(device)`.
- `forward()` — defines what *you* mean by "run the model forward". PyTorch auto-builds the backward pass.

The same model, the quick way — `nn.Sequential`

When the network is just a straight stack of layers, you don't need to write a class at all. `nn.Sequential` chains modules and runs them in order:

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 16),   # 3 inputs → 16 hidden
    nn.ReLU(),          # activation — now an explicit layer, not a function call
    nn.Linear(16, 1),   # 16 hidden → 1 output (logit)
)
```

This is **exactly** the `TurnoutModel` from the previous slide — same layers, same forward pass, same parameters:

$$\underbrace{3 \times 16 + 16}_{\text{layer 1}=64} + \underbrace{16 \times 1 + 1}_{\text{layer 2}=17} = \mathbf{81} \text{ parameters}$$

The differences are cosmetic

- `torch.relu(...)` (a function) becomes `nn.ReLU()` (a layer object)
- There is no `forward()` to write as `Sequential` calls each module in order.

Use `nn.Module` when the forward pass branches or reuses and `nn.Sequential` for simple, feedforward stacks.

A complete training script in 20 lines

```
model     = TurnoutModel()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn   = nn.BCEWithLogitsLoss()

for epoch in range(50):
    for x_batch, y_batch in train_loader:
        optimizer.zero_grad()
        y_logits = model(x_batch)                       # (B, 1) — a column
        loss = loss_fn(y_logits.squeeze(-1), y_batch)   # y_batch is (B,) — a row
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch:02d}: train loss = {loss.item():.3f}")
```

Every PyTorch training loop in the world looks roughly like this.

We'll change what's *inside* the loop but the shape of this training protocol stays the same.

WHY `.squeeze(-1)`? The last layer is $16 \to 1$, so a batch of 32 comes out as shape `(32, 1)`. The labels arrive as `(32,)` — i.e. a flat vector.

- This is annoying, but we need to "squeeze" the output so it is also `(32,)`.

The same pattern, in plain English

1. **Define a model** (a `Module` whose `forward()` you wrote).
2. **Pick a loss function** (cross-entropy, MSE, ...).
3. **Pick an optimiser** (Adam, SGD, ...).
4. **For each batch in your training data**:
    a. Zero the gradients.
    b. Forward pass.
    c. Compute the loss.
    d. Backward pass.
    e. One optimiser step.

Activation functions — what they actually do

A QUICK REMINDER WHY WE NEED NON-LINEARITIES

Without non-linearities, stacking layers buys us nothing:

$$\mathbf{x}\mathbf{W}^{(1)\mathsf{T}}\mathbf{W}^{(2)\mathsf{T}}\cdots\mathbf{W}^{(L)\mathsf{T}} = \mathbf{x}\mathbf{W}^{*\mathsf{T}}$$

— still just a linear function of $\mathbf{x}$, regardless of L.

Non-linear activations between layers are what gives deep learning its advantage

- They let the model bend, kink, and curve, not just separate with a single hyperplane.

ReLU — the modern default

$$\sigma_{\mathrm{ReLU}}(x) = \max(0, x), \qquad \sigma'_{\mathrm{ReLU}}(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

- Cheap.
- No vanishing gradient on the positive side.
- Sparse activations — many nodes output exactly 0.

The downside

Dead ReLUs. If a neuron's pre-activation stays ≤ 0, its gradient is always 0 — it stops learning.

Smoother variants (Leaky ReLU, GELU, SiLU) address this.

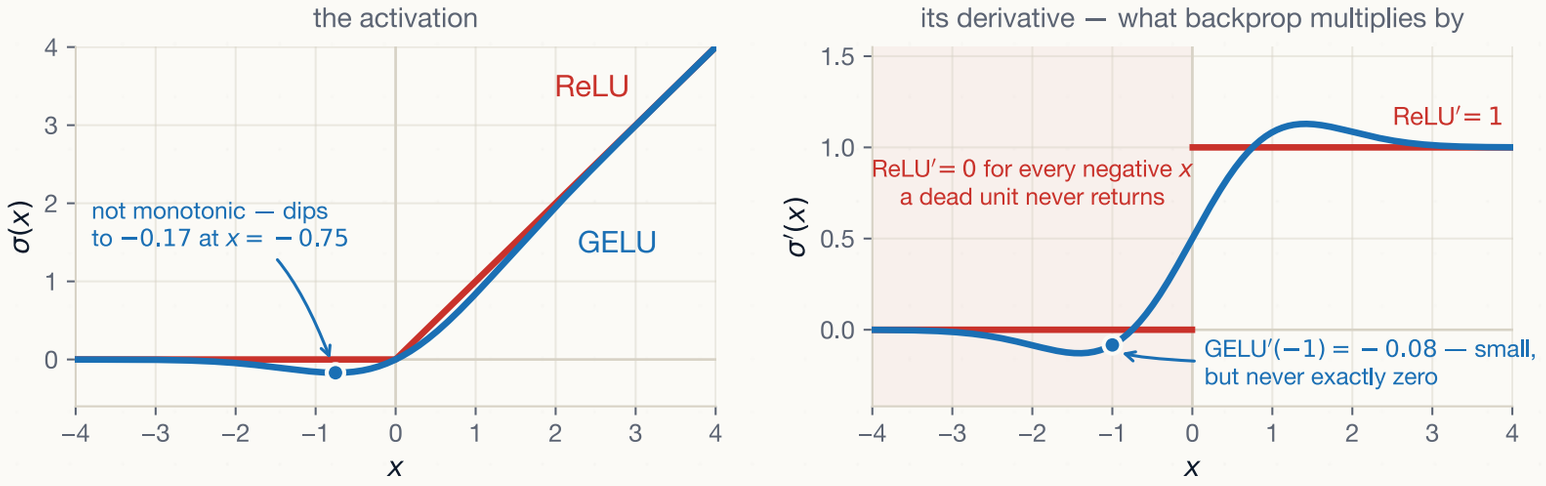
Default choice in 2026: use ReLU (or close cousin GELU) in hidden layers.

The overwhelming majority of feed-forward networks built in research do this. PyTorch: `nn.ReLU()`.

ReLU vs GELU — the same kink, smoothed

$$\sigma_{\mathrm{ReLU}}(x) = \max(0, x) \qquad \sigma_{\mathrm{GELU}}(x) = x \cdot \Phi(x)$$

Φ is the **standard-normal cumulative distribution function**. GELU is just a softer version of ReLU.



Since backprop multiplies by σ', ReLU′ is a step – exactly 0 for every negative input. GELU′ is never exactly 0: at $x = -1$ it is -0.083.

Never really worth using GELU on a simple network, but very useful for transformers as we'll see later!

Sigmoid and tanh

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x))$$

- Squashes to $(0, 1)$ — natural for **binary probabilities**.
- Used at output of binary classifiers.
- In hidden layers: **vanishing gradients** (flat tails).

Tanh

$$\tanh(x), \quad \tanh'(x) = 1 - \tanh^2(x)$$

- Squashes to $(-1, 1)$ — zero-centred.
- Better than sigmoid for hidden layers, but still saturates.
- Used in RNNs (Day 9!) and a few specialised places.

Identity, softmax — the output choices

IDENTITY

$$\sigma(x) = x$$

- For **regression** outputs — when y lives on the whole real line.
- Never in hidden layers (except specialised cases).

SOFTMAX

$$\sigma_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Squashes a vector of k scores into a probability distribution over k classes.
- Used at output of **multi-class classifiers**.
- Tied to **cross-entropy loss**.

WORKED: SOFTMAX OF $\mathbf{z} = [2, 1, 0]$

Exponentiate each score, then divide by their sum:

$$e^2 = 7.39, \quad e^1 = 2.72, \quad e^0 = 1.00, \quad \sum = 11.11$$

$$\sigma(\mathbf{z}) = \left[\tfrac{7.39}{11.11}, \tfrac{2.72}{11.11}, \tfrac{1.00}{11.11}\right] = [0.67,\ 0.24,\ 0.09]$$

Three positive numbers that sum to 1, i.e. a probability over three classes

A worked social-science example — civil-war forecasting

Hegre et al. (2022) — *Forecasting fatalities*.

- Predict country-month battle-deaths from a rich set of features.
- A challenging benchmark in conflict studies.
- The reference model is a deep feed-forward network.

The input vector: country indicators, lagged casualties, conflict history, geographic features, economic indicators.

The output: a single number — the predicted log fatalities for the next month.

A reasonable architecture:

- Input: ~30 features.
- Hidden 1: 128 → ReLU → dropout 0.2.
- Hidden 2: 64 → ReLU → dropout 0.2.
- Output: 1 (identity).

Train with **MSE** loss and **Adam** at $\lambda = 10^{-3}$ for ~100 epochs.

This is exactly the kind of model you'll build in today's lab.

Plenary · predict what each activation does

Plenary · predict the activations

≈ 10 MIN · IN PAIRS

THE PRE-ACTIVATION VALUE:

$z = -3.5$

For each activation, compute (or estimate) the post-activation value $\sigma(z)$:

1. **Identity** — $\sigma(z) = z$
2. **ReLU** — $\sigma(z) = \max(0, z)$
3. **Sigmoid** — $\sigma(z) = 1/(1 + e^{-z})$
4. **tanh** — $\sigma(z) = (e^{2z} - 1)/(e^{2z} + 1)$

Then for each, give the **derivative** at $z = -3.5$.

- Why does it matter that ReLU gives a 0 derivative and identity gives a derivative of exactly 1, while sigmoid/tanh give tiny derivatives?

Plenary · answers

Activation	$\sigma(-3.5)$	$\sigma'(-3.5)$
Identity	-3.5	1
ReLU	0	0
Sigmoid	≈ 0.029	≈ 0.028
tanh	≈ -0.998	≈ 0.004

THE LESSON

On the negative side at $|z| = 3.5$:

- **Identity** passes through, with derivative 1 → great for gradient flow.
- **ReLU** zeroes out — and its gradient is also zero on this example. If the unit stays negative for *every* input, it becomes a *dead ReLU* and stops learning permanently.
- **Sigmoid / tanh** are nearly flat — derivative $\to 0$. **Vanishing gradient.** Backprop through many such nodes shrinks gradients to nothing.

That is *why* most modern hidden layers use ReLU.

Part 2 · Training, properly

Loss functions, by problem type

Problem	Output activation *(applied inside the loss — don't add it yourself)*	Loss in PyTorch
Regression (continuous)	identity	`nn.MSELoss`
Binary classification	sigmoid	`nn.BCEWithLogitsLoss`
Multi-class classification	softmax	`nn.CrossEntropyLoss`
Multi-label	sigmoid per class	`nn.BCEWithLogitsLoss`

Tip

`BCEWithLogitsLoss` and `CrossEntropyLoss` **bundle** the sigmoid/softmax with the loss so *don't apply the sigmoid activation yourself*. This combined version is more stable.

Initialisation — why “all zeros” fails

PREDICT

Suppose we set *every* weight to exactly 0 before training. What goes wrong on the first step — and could training ever recover?

ANSWER — THE SYMMETRY TRAP

If every weight starts at 0:

- Every node in a layer gets the same input × same weights → the **same output**.
- Every node gets the **same gradient**, so every node **updates identically**.
- The network is effectively **one node wide** — symmetry never breaks, no matter how long you train.

Random initialisation breaks the symmetry. But the *scale* matters too: too big and activations explode; too small and they vanish — the same vanishing/exploding problem we saw with sigmoid, now driven by the *weights*, not the activation.

Xavier and He initialisation

Xavier (Glorot)

For sigmoid / tanh:

$$W^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n^{(l-1)}+n^{(l)}}\right)$$

Keeps the variance of activations stable across layers.

When the layer's fan-in and fan-out are similar this reduces to the fan-in (LeCun) form $\frac{1}{n^{(l-1)}}$.

HE (KAIMING)

For ReLU:

$$W^{(l)} \sim \mathcal{N}(0, \frac{2}{n^{(l-1)}})$$

The factor of 2 compensates for the half of the distribution ReLU zeroes out.

You won't usually set this by hand.

PyTorch's `nn.Linear` defaults to a Kaiming-uniform variant. You should know this is happening but only override it if you have a reason.

Optimisers — what we'll actually use

- **SGD** — vanilla. Rarely best in practice.
- **SGD with momentum** — better, still simple.
- **Adam** — adaptive learning rates per parameter. **Default first try.**
- **AdamW** — Adam with corrected weight decay. Default for transformers.

LEARNING RATE

- Most important hyperparameter you'll set.
- Reasonable starting points:
  - Adam: $\lambda = 10^{-3}$
  - SGD: $\lambda = 10^{-2}$
- Too high → loss explodes.
- Too low → loss crawls.

Modern practice

Use Adam with $\lambda = 10^{-3}$, then tune as needed.

The full training loop, revisited

```
import torch
from torch.utils.data import DataLoader

model     = MyModel()                                    # nn.Module
loss_fn   = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
val_loader   = DataLoader(val_set,   batch_size=64)

for epoch in range(num_epochs):
    model.train()
    for x_batch, y_batch in train_loader:
        optimizer.zero_grad()
        y_pred = model(x_batch)
        loss   = loss_fn(y_pred, y_batch)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        val_loss = sum(loss_fn(model(x), y) for x, y in val_loader) / len(val_loader)
```

What’s new in this version

- `model.train()` / `model.eval()` — toggles dropout and batch-norm behaviour (we’ll see why in Part 3).
- `torch.no_grad()` — disables gradient tracking during evaluation (saves memory and time).
- **Validation loss** — computed every epoch on held-out data.

⚠ **Never compute training and validation loss without toggling train/eval**

During training, several neural network features will operate differently than during testing (we’ll introduce some of these soon).

Activity · adjust one toggle, predict the effect

Activity · predict the effect

≈ 8 MIN · IN PAIRS

For each change, **predict** what happens to (a) training loss and (b) validation loss, then justify:

1. You **increase the learning rate** from 10^{-3} to 10^{-1}.
2. You **add a third hidden layer** (200 nodes wide).
3. You **drop the activation function** from all hidden layers — set σ to identity everywhere.
4. You **freeze the first hidden layer** — it doesn't update during training.

Activity · answers

1. **Higher LR.** Training loss might *explode* — overshoots minima. If it doesn't explode, it may converge faster but more noisily. Validation loss tends to mirror.
2. **Deeper model.** Train loss → lower (more capacity to fit training data). Val loss → uncertain depending on data and model specification (more on this later).
3. **No activations.** Both losses suffer dramatically. The model is now linear regression with extra parameters — same hypothesis class, more parameters to wiggle. Converges to the optimal linear fit.
4. **Frozen first layer.** With random init, the first layer is essentially a fixed random projection. Sometimes still works ("random kitchen sinks"), but typically losses both rise vs. the unfrozen model.

Part 3 · When training fails

Stop & check before we go on

STOP & CHECK

Before we go on to understand when models break, make sure you're happy with the following:

1. Why does **initialising every weight to zero** stop the network from learning?
2. What does `model.eval()` change about dropout and batch norm?
3. Which optimiser and learning rate are the sensible **first try**?

If any of these is still fuzzy, flag it now!

Neural networks are fickle

Even with the right loss, the right optimiser, and the right init, training can:

- **Diverge** — loss explodes to infinity.
- **Stall** — loss stops decreasing too early.
- **Overfit** — train loss → 0, validation loss climbs.
- **Underfit** — neither loss falls usefully.

> ⚠ **Warning**
>
> **You will encounter all of these** in the labs this week. The point of this section is to give you the vocabulary to *diagnose* what's happening — not to fix everything in one go.

Problem 1 — overfitting

Neural networks have many parameters. They can **memorise** the training set if you let them.

- Training loss → 0.
- Validation loss starts climbing.
- The model has fit *noise* in the training data, not signal.

> **Tip**
>
> The diagnostic is the gap between training and validation loss curves — watch them every epoch.

Defences

- Hold out a validation set.
- **Early stopping** — stop when val loss bottoms out.
- **Dropout** — random masking.
- **Weight decay** — penalise large weights.
- **More data** — when available.
- **Smaller model** — when you can't get more data.

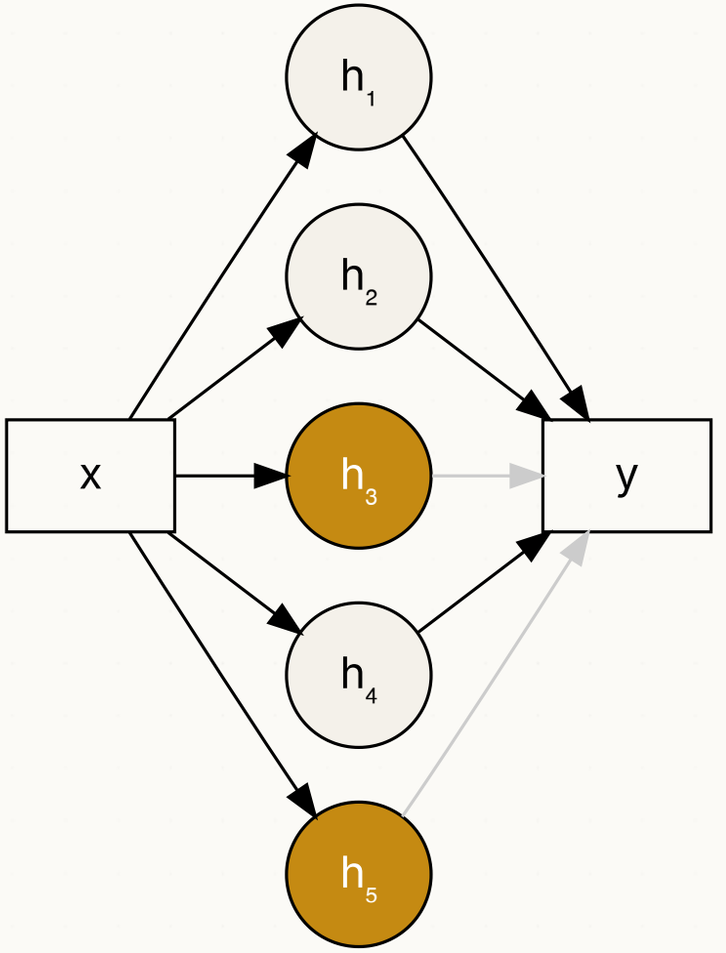
Dropout

During training, with probability p, **zero out** each node's output:

- Forces the network to learn **redundant representations**.
- Acts like an implicit **ensemble** of thinned sub-networks.
- Standard: $p = 0.2$–0.5 on hidden layers; **never** on the output.
- In PyTorch: `nn.Dropout(p=0.2)` after an (activated) layer.

DROPOUT IS TURNED OFF AT INFERENCE

- `model.eval()` scales the learned weights as a result

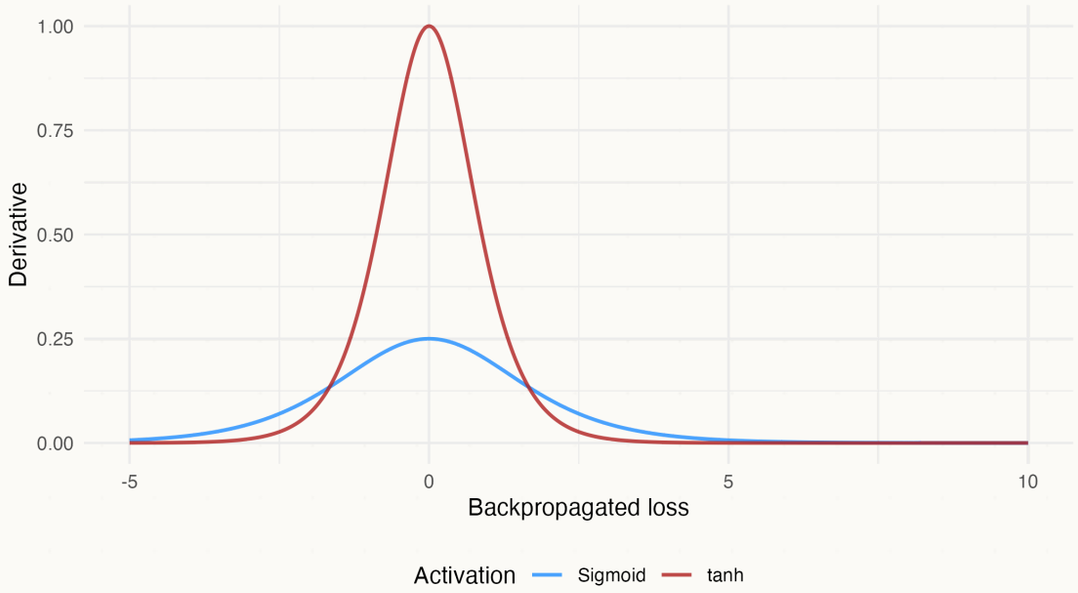


Amber units are the ones dropped on *this* step — a fresh random set every batch.

Problem 2 — vanishing gradients

For sigmoid / tanh:

- Their derivatives are nearly zero for large $|z|$.
- Backprop multiplies many derivatives via the chain rule.
- In a deep network this product results in **shrinking gradients**.
- Gradients near the input layer effectively vanish.
- Early layers stop learning.



FIXES

1. **Use ReLU-family activations** (no flat tail for $z > 0$).
2. **Normalise** activations across the network → next slide.

Batch normalisation — the idea (Ioffe & Szegedy, 2015)

Every layer learns against a moving target: as the layers *before* it update, the distribution arriving at this one shifts.

THE FIX After a layer's pre-activation, **rescale each feature so that — across the examples in the mini-batch — it has mean 0 and variance 1.**

$$\mathrm{BN}(z) = \gamma \cdot \underbrace{\frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}}_{\text{standardise}} + \beta$$

μ_B, σ_B^2 — mean and variance of **this one feature**, over the examples in the batch. γ, β are learnable scale and shift parameters, one pair per feature.

STOP & CHECK

A batch of **32** examples enters a hidden layer of width **64**. How many means μ_B does batch norm compute?

ANSWER

64 — one per *feature*, each averaged over the 32 examples. The batch is the dimension we average **away**.

Batch normalisation example

Take one feature's pre-activations across a batch of three examples: $z = [2,\ 4,\ 6]$, with $\gamma = 1$ and $\beta = 0$.

STEP 1 · THE BATCH MEAN $$\mu_B = \frac{2+4+6}{3} = 4$$

STEP 2 · THE BATCH VARIANCE $$\sigma_B^2 = \frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{3} = \frac{4+0+4}{3} = \frac{8}{3} \approx 2.67, \text{ so } \sigma_B \approx 1.63$$

STEP 3 · STANDARDISE $$\mathrm{BN}(z) \approx \left[\tfrac{2-4}{1.63},\ \tfrac{4-4}{1.63},\ \tfrac{6-4}{1.63}\right] = [-1.22,\ 0,\ 1.22]$$

- Because γ, β are *learned*, the network can **undo** this if the original scale was useful!

WHY IT HELPS

Stabilises activations across layers, stops distributions drifting during training, makes initialisation less critical, lets you use a larger learning rate.

⚠ At test time there is no batch to average over, so batch norm switches to **running averages** collected during training. That is one of the two things `model.eval()` toggles — and why forgetting it changes your numbers.

Layer normalisation

SAME ARITHMETIC. DIFFERENT DIRECTION.

Batch norm averages **down a column** — one feature, across the examples in the batch. Layer norm averages **along a row** — one example, across its own features.

$$\mathrm{LN}(h) = \gamma \cdot \frac{h - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}} + \beta$$

μ_L, σ_L^2 — the mean and variance of the d features **of this single example**. Nothing is shared across the batch.

WORKED: ONE EXAMPLE'S ACTIVATIONS $h = [2,\ 4,\ 6,\ 8]$

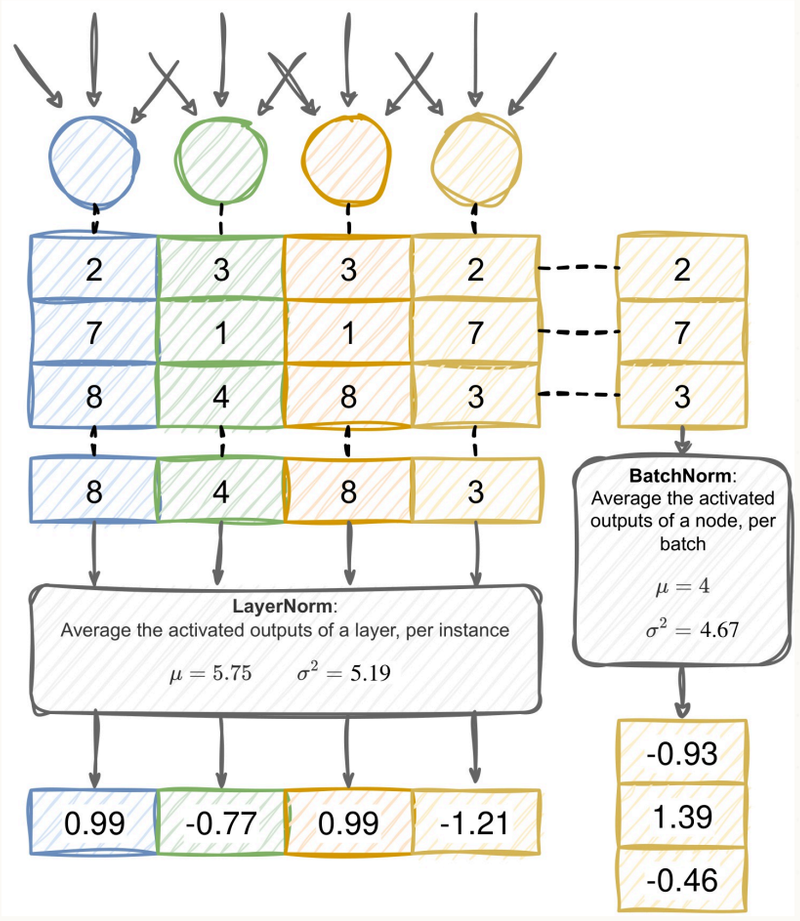
$$\mu_L = \frac{2+4+6+8}{4} = 5;\ \sigma_L^2 = \frac{9+1+1+9}{4} = 5, \text{ so } \sigma_L \approx 2.24$$

$$\mathrm{LN}(h) \approx \left[\tfrac{-3}{2.24},\ \tfrac{-1}{2.24},\ \tfrac{1}{2.24},\ \tfrac{3}{2.24}\right] = [-1.34,\ -0.45,\ 0.45,\ 1.34]$$

WHY SEQUENCES NEED IT

The statistics come from **one example**, so layer norm is *independent of batch size*. This will be useful for transformer (Day 10).

Batch vs layer norm

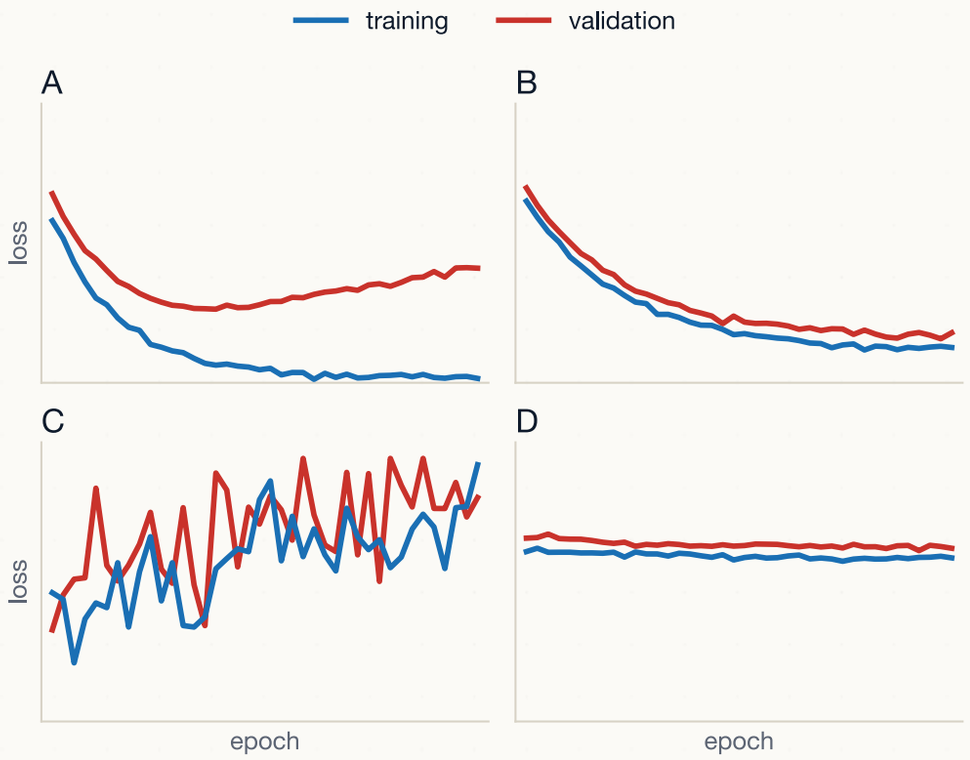


Batch norm one μ per **feature**, averaged down the batch. Needs a decent batch size. Predominantly used in **CNNs**.

LAYER NORM one μ per **example**, averaged across its features. Batch-size independent. Predominantly used in **transformers**.

Diagnosing a run from its loss curves

≈ 3 MIN · IN PAIRS



MATCH EACH PANEL TO A DIAGNOSIS

Four training runs. Same model, same data — four different outcomes:

- learning rate too high
- overfitting
- underfitting
- healthy

Which is which — and what in the curves told you?

These are the curves you plot in today's lab.

Diagnosing a run · answers

- **A — overfitting.** Training keeps falling; validation bottoms out, then climbs. *Fix:* dropout or weight decay, early stopping, or more data — the defences from the overfitting slide.
- **B — healthy.** Both curves fall together and the gap stays small. *Fix:* none. Leave it alone.
- **C — learning rate too high.** The loss jumps around and drifts upwards. *Fix:* lower the learning rate — from 10^{-1} back towards 10^{-3}.
- **D — underfitting.** Both curves flat and high from the first epoch. *Fix:* a bigger model, or train for longer.

One habit to keep

When a run misbehaves, plot these two curves before touching anything else. The shape tells you *which* problem you have — and each problem has a different fix.

A practical checklist for week 2 onwards

Always

- Normalise input features.
- Split: train / val / test, **never peek** at test.
- Track training **and** validation loss per epoch.
- Save model checkpoints.
- Set and log random seeds.

Most of the time

- Adam, $\lambda = 10^{-3}$ as a starting point.
- ReLU + He init for hidden layers.
- Dropout 0.2 in fully-connected layers.
- Batch norm between layers (CNNs); layer norm (sequences).
- Increase model size *gradually*.

Take stock

WHERE YOU ARE AT THE END OF DAY 5

You can:

1. Write a working feed-forward network in PyTorch.
2. Pick a sensible loss, optimiser, and learning rate.
3. Diagnose overfitting, vanishing gradients, and dead ReLUs from training curves.
4. Apply dropout and batch norm when appropriate.
5. Apply the same machinery to a real social-science problem like civil-war forecasting.

This is the modal neural network used in industry and research for tabular data. From tomorrow we adapt it for images and text.

A check-in on the headline promise

WHERE WE ARE ON THE ROAD TO BUILDING A GPT

Where we are right now, end of Day 5:

- ✅ You can write a forward pass.
- ✅ You can write backprop (by hand, then in PyTorch).
- ✅ You can train a feed-forward classifier on real data.
- ⏳ Next week: vision (CNNs, generative).
- ⏳ Week 3: language. Embeddings, attention, transformers — your tiny GPT.

A third of the way in

Lab brief · today’s afternoon

Today’s lab — 90 minutes

GOALS

In Colab (or local), build a small feed-forward classifier on a real social-science dataset.

You will:

1. Load data and create train / validation / test splits.
2. Define a model with `nn.Sequential` (linear → ReLU → linear → ReLU → linear).
3. Train with `Adam` + cross-entropy.
4. Plot loss curves.
5. Add **dropout** and **batch norm**. Re-train. Compare.

Stretch goals

- Try replacing `ReLU` with `Sigmoid`. Watch what happens to the loss curve.
- Try a learning rate of 10^{-1}. Watch what happens to the loss.
- Increase model depth to 4 layers. Watch validation accuracy.

See you next week

Tip

#Optional reading

Torres & Cantú (2022) — *Learning to See: Convolutional Neural Networks for the Analysis of Social Science Data*. A tour of CNNs from a social-science perspective.

Hegre, H., Akbari, F., Croicu, M., Dale, J., Gåsste, T., Jansen, R., Landsverk, P., Leis, M., Lindqvist-McGowan, A., Mueller, H., et al. (2022). *Forecasting fatalities*.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning*, 448–456.

Torres, M., & Cantú, F. (2022). Learning to see: Convolutional neural networks for the analysis of social science data. *Political Analysis*, *30*(1), 113–131.